SCHOLARSHIP INFRASTRUCTURE ON BOT CHAIN

SchoChain

Transparent Scholarship Management and Funding on BOT Chain

A wallet-based platform for foundations, schools, organizations and sponsors to fund scholarship pools, review applicants, publish decisions and pay awards - with public rules, reserved funds and minimal personal data.

Stack	HTML5, CSS3, vanilla JavaScript, Solidity 0.8.24, ethers.js v6. No backend, database or centralized storage.
Network	BOT Chain (EVM). Testnet chain ID 968, mainnet chain ID 677.
Status	Reference implementation. Unaudited. See Section 9 before using with real funds.

1. Summary

Scholarship programs depend on trust: applicants trust that rules are applied fairly, and funders trust that money reaches the intended students. SchoChain moves the parts of that process that benefit from public verification onto BOT Chain - program rules, the funded pool, the application record, review counts, decisions and payments - while deliberately keeping personal information off the chain.

Each scholarship is its own smart contract with its own treasury. A provider funds it in native BOT or USDT, authorizes reviewers, and records decisions. Approving an application reserves the award from the pool at that moment, so the program can never promise more than it holds or more awards than its cap. Students claim approved awards to their own wallets within a claim window; unclaimed or surplus funds return to the provider only after the program is closed or cancelled.

Important limitation. A blockchain is public and permanent. SchoChain therefore records only wallet addresses, application status, public eligibility declarations and document hashes. A document hash is not privacy and not storage. Section 5 explains exactly what that means.

2. Roles and permissions

Role	How they are identified	What they can do
Platform admin	Owner of ProviderRegistry and ScholarshipFactory (two-step ownership transfer)	Verify, suspend or reinstate providers; toggle the verification requirement; pause new program creation; set the USDT token address for new programs. **Cannot** move scholarship funds, change a decision or block student claims.
Provider	Wallet registered in ProviderRegistry and verified by the admin	Create programs; fund pools; authorize/remove reviewers and assign them to applications; record, revise (until published) and publish decisions; disburse; expire unclaimed awards; close or cancel; withdraw surplus.
Reviewer	Wallet authorized by a provider for one program	See assigned applications; submit one score (0-100), a recommendation and a notes hash per application. Cannot decide, fund or withdraw.
Student	Any wallet that is not the provider or a reviewer of that program	Apply once per program; withdraw before a decision; track status; claim an approved, published award; view history.

3. Architecture and smart contracts

SchoChain is four production contracts plus a testnet token. The program contract is deployed once as an implementation and cloned (EIP-1167 minimal proxy) by the factory, so every scholarship has isolated state and funds at low deployment cost.

Contract	Responsibility
ProviderRegistry	Scholarship provider registry. Organizations register public name and link; the admin verifies, suspends or reinstates. Gate for program creation.
ScholarshipFactory	Scholarship program factory. Validates the caller is a verified provider, resolves the currency (BOT or configured USDT), deploys and initializes a program clone, and indexes programs by provider.
ScholarshipProgram	One per scholarship. Contains: the **scholarship treasury** (fund, accounting), **reviewer authorization** and assignment, the **application book**, the **decision and award manager**, the **student claim and disbursement manager**, and **refund and cancellation logic**.

Contract	Responsibility
ApplicationRegistry	Global index so a student's dashboard can list every (program, application id) for their wallet, and a reviewer's dashboard can list programs they serve. Only factory-created programs can write to it.
MockUSDT (testnet only)	Six-decimal token with open minting for testing. Never deploy to mainnet; the deploy script refuses without a real USDT address.

Requirement-to-contract mapping

Requested component	Where it lives
Provider registry	ProviderRegistry
Program factory	ScholarshipFactory
Application registry	ScholarshipProgram (application book) + ApplicationRegistry (cross-program index)
Reviewer authorization	ScholarshipProgram: addReviewer, removeReviewer, assignReviewers
Decision and award manager	ScholarshipProgram: decide, publishDecisions, expireAward
Scholarship treasury	ScholarshipProgram: fund, fundToken, committed / available accounting
Claim and disbursement manager	ScholarshipProgram: claim (student pull), disburse (provider push)
Refund and cancellation	ScholarshipProgram: closeProgram, cancelProgram, withdrawSurplus

4. Scholarship workflows

4.1 Provider workflow

- **Register.** Submit organization name and public link to ProviderRegistry. Status is Pending until the admin verifies (or the admin disables the verification requirement on a test deployment).
- **Create a program.** Provide title, public summary, up to 16 eligibility criteria (each optionally required), currency, fixed award amount, maximum number of awards, application deadline, required reviews per decision (1-5) and claim window (7-365 days).
- **Fund.** Send BOT with fund(), or approve USDT and call fundToken(). Only the provider can fund; plain transfers to the contract revert so money cannot arrive outside the accounting.
- **Authorize and assign reviewers.** Reviewers are authorized per program and assigned per application. A reviewer cannot be assigned to their own application, and an applicant cannot be added as a reviewer.
- **Decide.** Once the review quorum is met, record approve or reject with a reason hash. Approval reserves the award immediately. Decisions may be revised until published; revising an approval releases its reservation.
- **Publish.** Publishing makes decisions official. Approved awards become claimable and their claim window starts; rejections are finalized.
- **Disburse or let students claim.** The provider can push a payment to the student's wallet, or the student can claim it. Either path pays at most once.
- **Close, cancel, refund.** See Section 6.

4.2 Student workflow

- Connect a wallet (the app switches or adds BOT Chain), browse the explorer and open a scholarship to see requirements, award, deadline and pool status.
- Apply once per wallet: tick the public eligibility declarations (all required ones must be ticked), choose supporting file(s) which are hashed locally in the browser, optionally add a short non-sensitive reference, and acknowledge the privacy notice.
- Track status on the student dashboard: Submitted, Under review, Approved, Rejected, Claimed, Withdrawn, Expired or Cancelled. Withdraw while undecided if needed.
- After a published approval, claim the award before the claim deadline. Previous applications and awards remain visible under history.

4.3 Reviewer workflow

- The provider authorizes the reviewer's wallet and assigns applications. The reviewer dashboard lists only assigned applications, with the applicant's wallet, public declarations and document hash.
- The reviewer submits a score from 0 to 100, a recommendation and a hash of private notes. Each reviewer may review each application once. Notes themselves are never recorded.

4.4 Application status model

Status	Meaning
Submitted	Recorded; no review yet.
Under review	At least one review received.
Approved	Decision published as approve; award reserved and claimable until the claim deadline.
Rejected	Decision published as reject.
Claimed	Award paid to the student's wallet.
Withdrawn	Student withdrew before a decision.
Expired	Approved award not claimed within the window; reservation released.
Cancelled	Program cancelled while the application was open or its approval unpublished.

5. Application privacy and its limits

The design rule is simple: **if it would be harmful to publish permanently, it never goes onchain.** Under the strict no-backend constraint there is nowhere private to put it, so SchoChain does not take it at all.

Recorded onchain (public)	Never collected by SchoChain
Student wallet address; program-local application id and a global application key	Name, date of birth, address, phone, email
Application status, review count, average score, decision flag and reason hash	Identification documents, transcripts, financial records
Public eligibility declarations (a bitmask over the program's published criteria)	Free-text essays or personal statements
A SHA-256 hash of the applicant's document bundle; optional short reference (max 160 bytes)	Reviewer notes (only a hash is recorded)

5.1 What a document hash does and does not do

A cryptographic hash is a fixed-size fingerprint of a file. It proves that a document existed in a specific form when the application was submitted, and lets a provider confirm that a file they later receive is identical. That is all it does.

A document hash does not make the document private, secure or stored.

- **Not storage.** The file is not saved anywhere by SchoChain. The student keeps it and sends it to the provider through a channel the provider specifies. If the student loses the exact file, the hash cannot be reproduced.
- **Not encryption.** A hash cannot be reversed, but anyone who already has the same file (or can guess its contents) can compute the hash and confirm a match. Low-entropy documents such as a short form or a standard certificate are guessable.
- **Not anonymity.** The wallet address is public and permanent. Anyone who links a wallet to a person can see that person's applications, declarations, statuses and payments. Students who want separation should use a dedicated wallet.
- **Not a secrecy window.** The optional reference field and all declarations are public forever. The interface warns users and the reference should stay empty unless it is clearly non-sensitive.
- **Decisions are public once recorded.** The publish step controls when a decision becomes official and claimable, and the interface hides unpublished decisions. But the DecisionRecorded event and the contract's read functions expose a recorded decision to anyone who looks. Publication is a workflow gate, not confidentiality.

5.2 Practical guidance

- Providers should collect documents off-chain through a channel that suits their data-protection obligations and verify hashes against it.
- Students should never type personal details in any text field and should use a wallet they are comfortable having publicly linked to a scholarship application.
- Providers remain responsible for compliance with privacy law in their jurisdiction. SchoChain reduces what is exposed; it does not remove obligations that arise from the off-chain document exchange.

6. Funding model and fund safety

Every program tracks four numbers, all in the program's own currency (native BOT with 18 decimals, or the configured ERC-20 with its own decimals):

Quantity	Definition
totalFunded	Sum of all provider deposits (for tokens, the amount actually received, so fee-on-transfer tokens cannot over-credit).
available	totalFunded - totalPaid - totalRefunded: money still held by the contract.
committed	Funds reserved for approved awards that are not yet paid or expired.
unallocated	available - committed: the only amount a provider may ever withdraw, and only after close or cancel.

6.1 Currencies

A program has one currency: BOT or USDT. A provider that wants to offer both creates two programs. The USDT address is set by the platform admin on the factory and snapshotted into each program at creation, so later changes to the factory setting cannot redirect an existing program. USDT decimals are read from

the token by the interface.

6.2 Lifecycle of funds

- **Deposit** increases totalFunded. Only the provider may fund and only while the program is not cancelled.
- **Approval** moves one award from unallocated to committed. It reverts if committed + award would exceed available, or if the maximum number of awards is already reserved or paid.
- **Payment** reduces committed and increases totalPaid before the transfer executes (checks-effects-interactions).
- **Expiry** (after the claim deadline) releases the reservation back to unallocated. The student can no longer claim it.
- **Close** requires the deadline to have passed and every application to have a recorded decision or be withdrawn. **Cancel** is available while active: it voids unpublished approvals and releases their funds, and marks open applications Cancelled. Awards already published remain claimable until their own deadline.
- **Surplus withdrawal** is only possible after close or cancel and only up to unallocated. Reserved awards can never be withdrawn by the provider.

7. Award distribution

Awards are a fixed amount per program, which keeps the over-allocation invariant simple and auditable. The distribution rules are enforced by the contract:

- **No duplicate awards.** One application per wallet per program; a wallet that withdrew cannot re-apply. Status moves forward only: a Claimed award cannot be claimed again, and an application has at most one award.
- **No over-allocation.** Approvals are capped by maxAwards and by funded money at the moment of approval. Reservations are released only by revising an unpublished decision, expiry or cancelling unpublished approvals.
- **No unauthorized decisions.** Only the program's provider can decide, publish, disburse, expire, close or cancel. Reviewers can only submit their own assigned review, once.
- **Quorum before decision.** A decision requires the provider-configured number of reviews. The quorum is a process safeguard: the provider still makes the final decision, and the review scores are advisory.
- **Student-controlled payout.** The award can only be sent to the applicant's own wallet, whether the student claims it or the provider disburses it. Neither can redirect funds elsewhere.

8. Security design

Area	Measures in this implementation
Access control	Per-program provider checks; per-program reviewer allow-list plus per-application assignment; platform admin limited to registry and factory settings; two-step ownership transfer; ApplicationRegistry writes restricted to factory-created programs.
Reentrancy	A reentrancy guard on claim, disburse, withdrawSurplus and token funding; all accounting updated before any external transfer.
Arithmetic	Solidity 0.8 checked arithmetic. Counters and reservations are adjusted in matching pairs and unit-tested.
Token handling	Safe ERC-20 calls that verify the token is a contract and tolerate tokens with no return value; credited amount measured by balance difference.
Clone safety	Implementation locked at construction; clones can be initialized once, by the factory path; factory registers the clone before initialization so the registry accepts its writes.
Funds isolation	One treasury per program: a defect in one program's data cannot drain another. No receive or fallback function, so stray transfers revert.
Provider suspension	A suspended provider cannot receive applications, fund, assign, decide or publish. Student claims and the provider's surplus withdrawal keep working so students are never trapped.
Input validation	Length limits on every string, range checks on deadlines, quorum, claim window and eligibility masks.
Front end	All onchain strings are HTML-escaped before display and links are restricted to http(s). No third-party scripts: ethers.js is bundled locally. Documents are hashed in the browser and never uploaded.
EVM compatibility	Compiled for the Paris EVM target (no PUSH0), the conservative choice for BSC-lineage EVM chains.

8.1 Test coverage

The Hardhat suite (29 tests) covers provider gating, factory validation, clone initialization, BOT and USDT funding, application rules, duplicate prevention, reviewer authorization and assignment, quorum, the maximum-awards cap, pool-funding over-allocation, decision revision, publish gating, single claims, provider disbursement, claim expiry, close/cancel/refund rules, suspension behavior and registry access control.

9. Limitations and risks

This is a reference implementation. It has not been independently audited and should not hold material funds until it has been. Known design trade-offs:

- **Provider liveness.** A provider who never decides leaves applicants waiting, and cannot close the program or withdraw surplus until every application has a decision. This protects students but can lock funds if a provider disappears. A governance or timeout mechanism is a natural extension.
- **Provider and reviewer trust.** Review scores are advisory; the provider decides. Collusion between a provider and reviewers cannot be prevented onchain. Transparency makes it visible, not impossible.
- **Sybil applicants.** One application per wallet does not prove one application per person. Providers must verify identity off-chain before disbursing or rely on the claim window after publishing.
- **Admin trust for verification.** Provider verification is performed by the admin wallet. Use a multisig for the admin on mainnet.

- **Fixed award and one currency per program.** Variable or tiered awards and multi-currency programs would need extra accounting.
- **Smart-contract wallets.** Native BOT payouts to contract wallets that reject incoming transfers will revert; the student can supply a different wallet by applying from it.
- **Chain and token assumptions.** Behavior depends on BOT Chain finality and on the USDT contract behaving like a standard ERC-20. Review the token before configuring it.

10. Deployment summary

Full instructions are in the project README. In short:

Step	Command / action
Install and test	npm install; npx hardhat test
Configure	Set PRIVATE_KEY (and USDT_ADDRESS for mainnet). Testnet RPC https://rpc.bohr.life (chain 968); mainnet RPC https://rpc.botchain.ai (chain 677).
Deploy	npx hardhat run scripts/deploy.js --network botTestnet. This deploys the registries, the program implementation and the factory, links the registry to the factory, and writes frontend/js/deployment.js.
Export ABI	npx hardhat compile; node scripts/export-abi.js
Host the front end	Serve the frontend folder with any static server (for example npx serve frontend). Wallets require http(s), not file://.
Go live	Register a provider wallet, verify it from the admin page, create a program, fund it, and invite reviewers and students.

11. Roadmap

- Provider-side timeouts and a dispute path for unresponsive providers.
- Variable award tiers and multi-installment disbursements tied to milestones such as enrollment confirmation.
- Optional encrypted document exchange between student and provider using wallet-derived keys, with ciphertext held off-chain.
- Multisig administration and an independent security audit before mainnet use.

Disclaimer: this document describes a software design for informational purposes. It is not legal, financial or data-protection advice.